西安電子科技大學

《计算机操作系统课程设计》

课 程 报 告

题　　目 实验二：Alarm-Clock

指导教师 李海昆

学　　院 计算机科学与技术学院　专　　业 计算机科学与技术

学　　号 19050500032　班　　级 1903011

姓　　名 许珈铭

完成时间 2021 年 12 月 23 日

目录

1 实验目的 **1**

2 实验原理 **1**

2.1 pintos 系统内核代码理解 1

2.2 基本原理 4

3 实验方案 **5**

3.1 环境配置 5

3.2 正式实验 9

3.3 实验结果 11

4 实验心得 **12**

实验二：Alarm-Clock

1 实验目的

1. 了解 pintos 操作系统 Alarm-clock 的基本实现原理。
2. 理解 threads.c、threads.h、interrupt.c、interrupt.h、time.c、time.h 中某些结构体、函数的含义与作用
3. 重新实现 timer_sleep() 函数，避免“忙则等待”的发生。

2 实验原理

2.1 pintos 系统内核代码理解

1. *../pintos/src/threads/* 目录下的 *thread.h* 和 *thread.c* 文件是有关线程初始化、阻塞、解除阻塞，线程调度等内容。
   可以通过输入 `geany thread.c` `geany thread.h` 命令用 *geany* 文本编辑器来打开相应的文件查看内容。当然也可以使用 *vim* 或 *gedit* 文本编辑器来替换 *geany*。

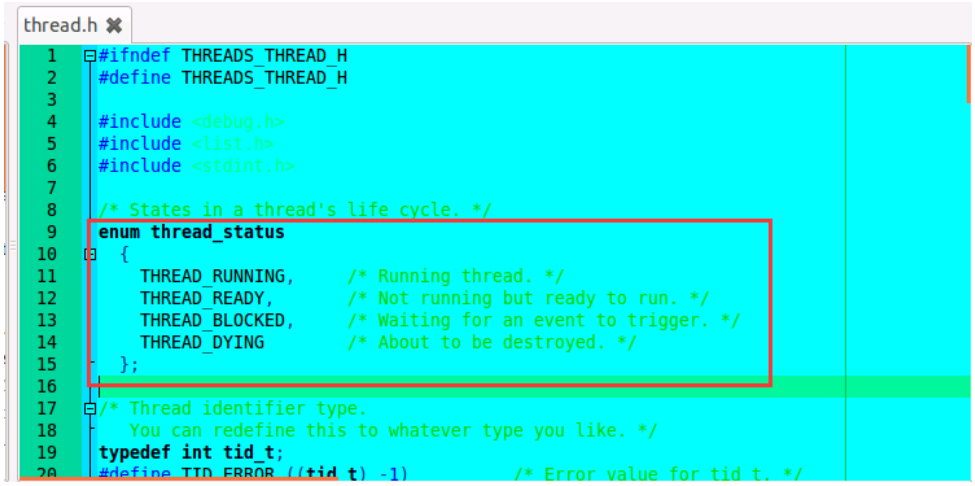


图 1: thread.h 文件

- 理解 *thread.h* 文件中的 *thread* 结构体。

```
struct thread
  {
    /* Owned by thread.c. */
    tid_t tid;                          /* Thread identifier. */
    enum thread_status status;          /* Thread state. */
    char name[16];                      /* Name (for debugging purposes). */
    uint8_t *stack;                     /* Saved stack pointer. */
    int priority;                       /* Priority. */
    struct list_elem allelem;           /* List element for all threads list. */

    /* Shared between thread.c and synch.c. */
    struct list_elem elem;              /* List element. */

#ifdef USERPROG
    /* Owned by userprog/process.c. */
    uint32_t *pagedir;                  /* Page directory. */
#endif

    /* Owned by thread.c. */
    unsigned magic;                     /* Detects stack overflow. */
  };
```

图 2: thread 结构体内容

tid	该线程的编号
thread_status	该线程目前的状态（enum 类型）
name[16]	该线程的名字
priority	该线程的优先级
...	...

- 其中 *thread_status* 的类型是 *enum*，*enum* 类型是 C 语言中的枚举类型。常见用法如下：

```
enum 标识符
{
    //枚举数据表
};
```

```
enum thread_status
  {
    THREAD_RUNNING,     /* Running thread. */
    THREAD_READY,       /* Not running but ready to run. */
    THREAD_BLOCKED,     /* Waiting for an event to trigger. */
    THREAD_DYING        /* About to be destroyed. */
  };
```

图 3: thread_status

所以由图可知，*thread_status* 只可以取 *THREAD_RUNNING,THREAD_READY, THREAD_BLOCKED,THREAD_DYING* 这些作为进程状态。

- *thread.h* 和 *thread.c* 文件中还有一些需要了解其作用的函数，如下表：

thread_current	获取当前线程的指针
*thread_foreach(thread_action_func *func, void *aux)*	遍历当前 ready queue 中的所有线程，并且对于每一个线程执行一次 func 操作
thread_block()	把当前占用 CPU 的线程阻塞掉（放到 waiting queue 里面）
thread_unblock()	将已经被阻塞掉的线程唤醒到 ready 队列中
...	...

2. *../pintos/src/threads/* 目录下的 *interrupt.c* 和 *interrupt.h* 文件中是与中断有关的处理函数。
   其中在这两个文件中包含了我们这次实验对象 Alarm-Clock 的驱动力 timer_interrupt 定时器中断函数。有了定时器中断函数，那么还需要定义定时器中断频率。

3. *../pintos/src/devices/* 目录下的 *time.h* 和 *time.c* 文件中包含了本实验要修改的 *time_sleep()* 函数，其中 *time.h* 文件中定义了定时器中断频率。

```
/* Number of timer interrupts per second. */
#define TIMER_FREQ 100
```

图 4: time-freq 定时器中断频率

由上图可知一个定时器中断时长大概是 10ms，这里称为一个 ticks。Pintos 中一个时间片的长度 =4*ticks=40 ms。当一个线程运行了一个时间片后，它必须放弃处理器给其它的线程。那么系统是如何知道线程运行的时间，以及何时对线程切换呢？其中 *time.h* 和 *time.c* 文件中还有一些需要了解其作用的函数，如下表：

timer_ticks()	返回自从系统启动经过的时间量 (以 ticks 为单位)。
timer_elapsed()	返回自某个时刻起经过的时间量 (以 ticks 为单位)
timer_sleep()	本次实验需要修改的函数之一
...	...

2.2 基本原理

1. 回顾 *timer_sleep()* 函数

```
/* Sleeps for approximately TICKS timer ticks.  Interrupts must
   be turned on. */
void
timer_sleep (int64_t ticks)
{
  int64_t start = timer_ticks ();

  ASSERT (intr_get_level () == INTR_ON);
  while (timer_elapsed (start) < ticks)
    thread_yield ();
}
```

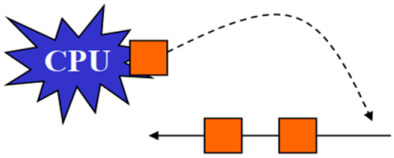


图 5: timer_sleep() 函数

- 考虑在 *timer_sleep()* 函数中让线程暂时阻塞 (*thread_block()*)，然后等待 ticks 个定时器中断周期结束以后再去唤醒它 (*thread_unblock()*)。
- “等”不再使用循环来实现,而是在每个定时器中断到来后去检查一下 *wait_list* 中的线程，看是否已经到了唤醒的时机。如果时机已到, 马上将其唤醒进入 *ready_list()*；如反之则不做任何操作。
- 没有了循环调用，其他线程就可以充分在 CPU 上运行。

2. 基本原理图

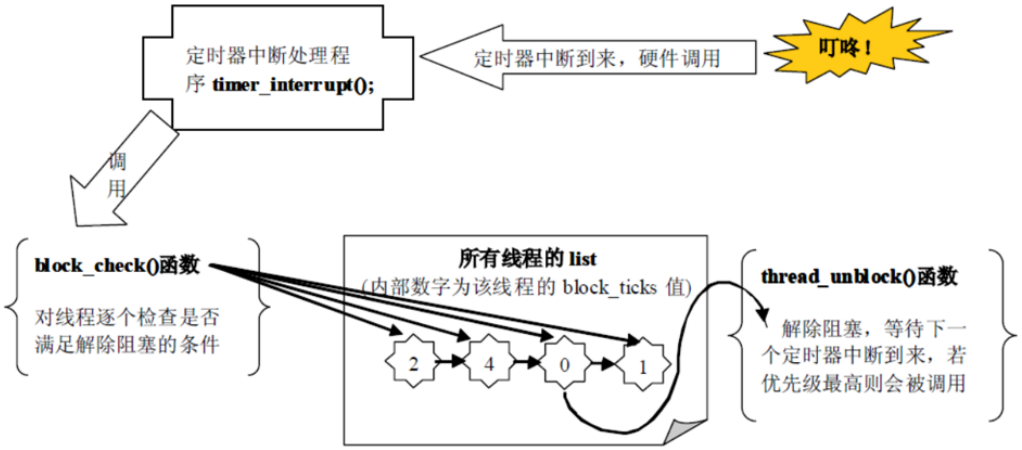


图 6: 基本原理图

3 实验方案

3.1 环境配置

3.1.1 云服务器的连接

- 实验工具：阿里云服务器、Xshell 远程连接软件、VNC Viewer。
- 实验步骤：按如下步骤执行即可

1. 打开 Xshell 软件，点击新建

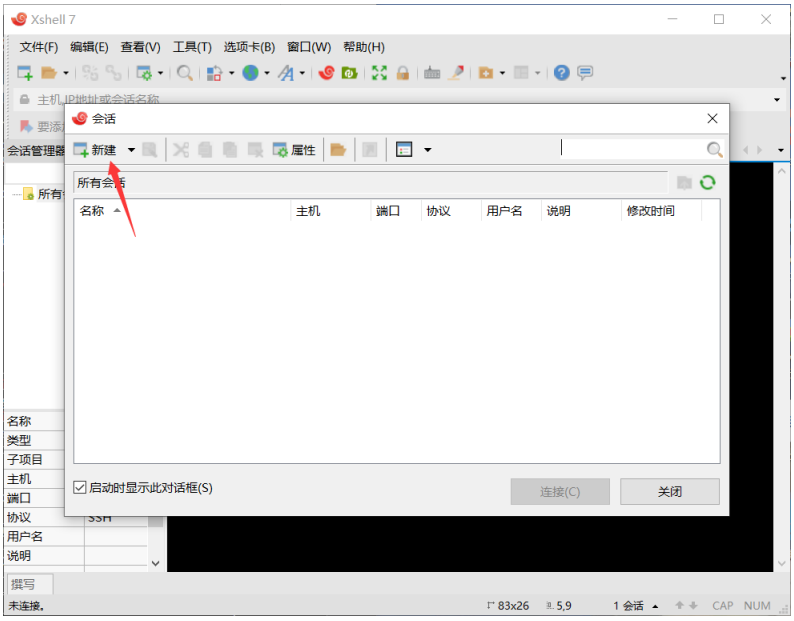


图 7: Xshell 软件界面-1

2. 在阿里云网站上申请相应的服务器资源后，按照图中指示顺序填写相关内容

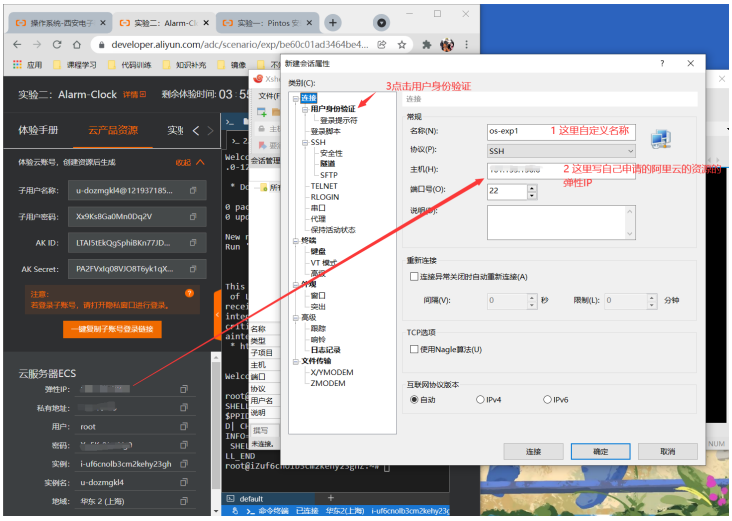


图 8: Xshell 软件界面-2

3. 然后点击用户身份验证，填入相应的身份表示信息

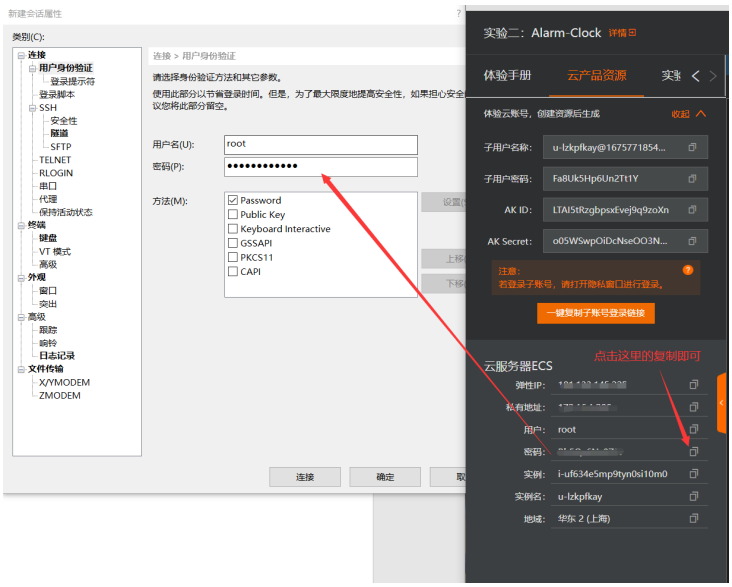


图 9: Xshell 软件界面-3

4. 出现这样的界面点击“接受并保存”，出现 *Welcome to aliyun Elastic Compute Sevice* 则表示远程连接成功了。

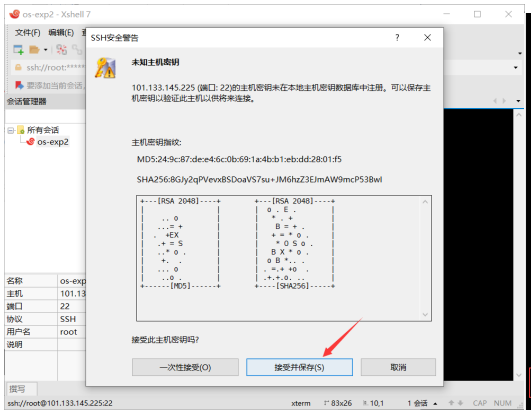


图 10: Xshell 软件界面-4

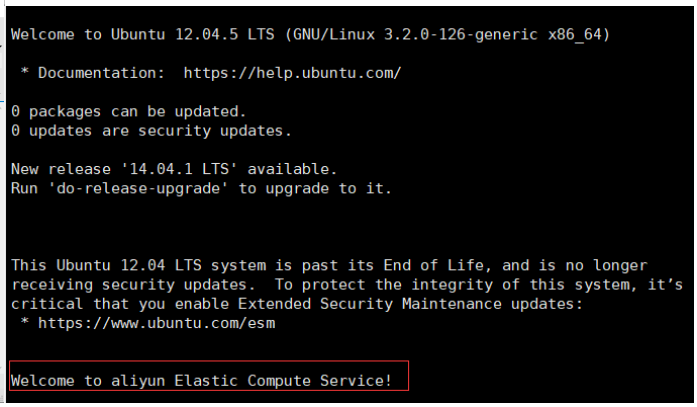


图 11: Xshell 软件界面-5

3.1.2 用 *VNC Viewer* 使用图形化界面登录

- 实验工具：*VNC Viewer*。
- 实验步骤：按如下步骤执行即可

1. 安装 *VNC Viewer* 后，打开软件，在相应的地址栏输入**弹性 IP 加上:1**

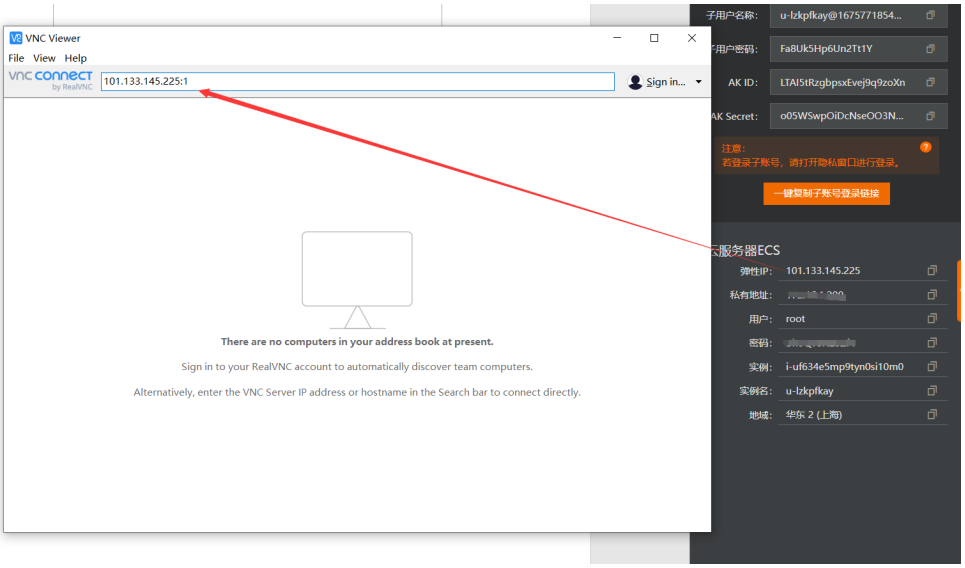


图 12: VNC 软件界面-1

2. 如果出现如下左图，直接点击 *continue* 即可显示右图，输入初始密码 *123456*

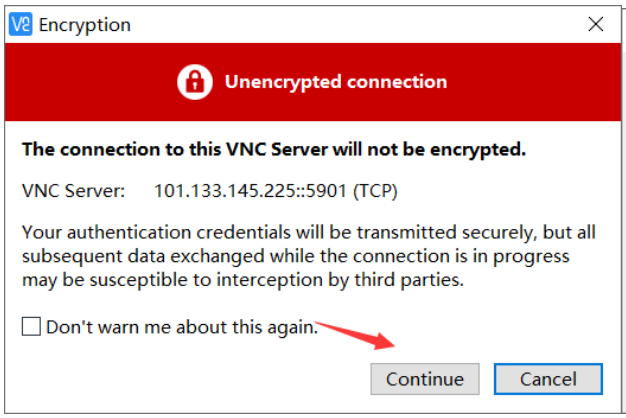


图 13: VNC 软件界面-2

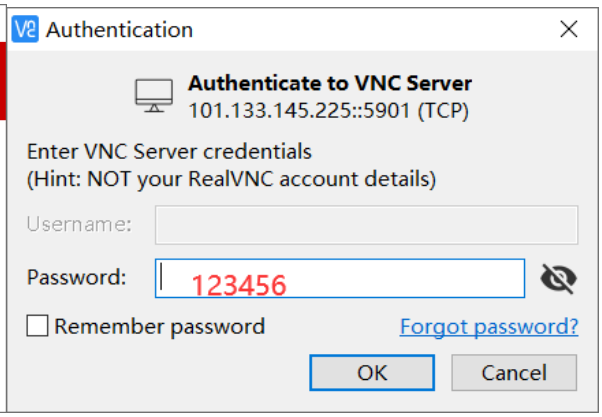


图 14: VNC 软件界面-3

3. 如果出现下图，一般这样的情况都是所连接的服务器没有配 *VNC* 的环境。

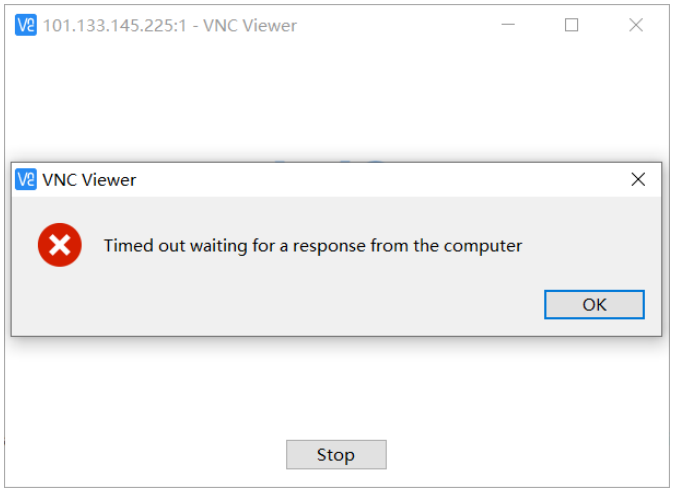


图 15: VNC 软件界面-4

所以需要先用 Xshell 按照 3.1.1 的云服务器的连接步骤连接服务器（Windows 用 Powershell 中的 SSH 也行，Mac 直接 Terminal 中 SSH 命令，本质都是一样的）进入命令行模式。在 *Xshell* 的命令行中依次输入:

- `vncserver :1`
- `vncserver -geometry 2560x1440`

注意:这里的‘x’是字母 x，不是‘*’，这里是调整分辨率，可以根据你的电脑显示屏的分辨率来调，可以低于电脑分辨率，不能高于分辨率，我这里的显示器是 2k 屏，所以是 2560x1440,1080P 的话应该就是 1920x1080 不然可能会出现颜色乱码。

注意:结束实验后，服务器会自动清除相关配置，所以在每一次体验创建资源后，都需要配置一遍 *VNC Viewer*，不然就会出现上图的 *Timed out*

4. 我们成功进入图形化界面, 我们的基本配置也算是结束了。

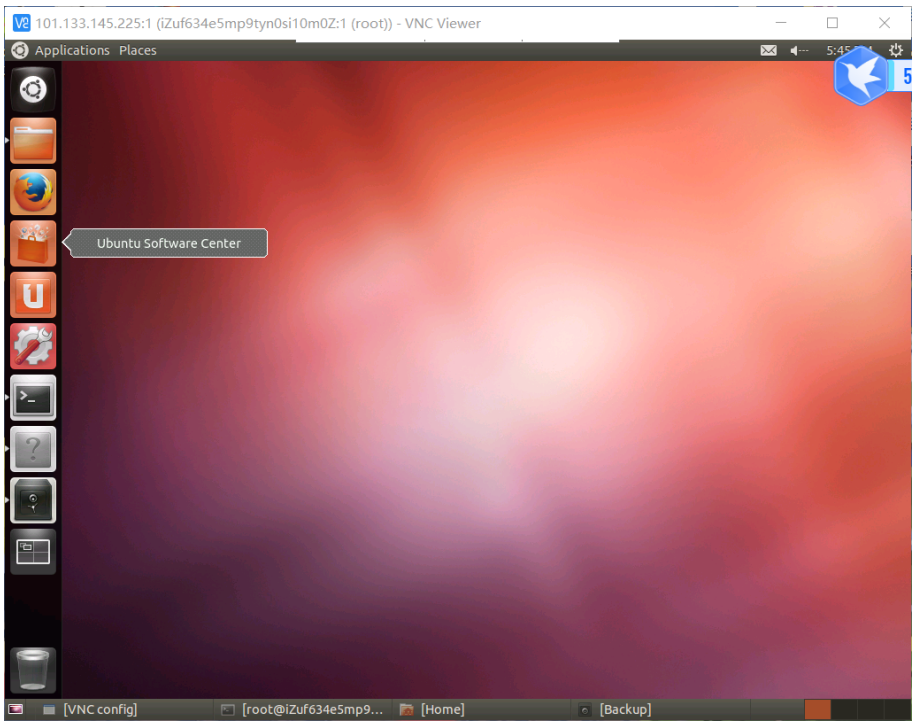


图 16: VNC 软件界面-5

注意：因为这是在云服务器上完成的，所以服务器已经给我们预装了 *bochs* 虚拟机以及安装在其中的 *pintos* 系统。可以在对应的*/home* 下文件夹下查看。如果没有下图的内容，则需要自己安装 *bochs* 和 *pintos*。

图 17: VNC 软件界面-6

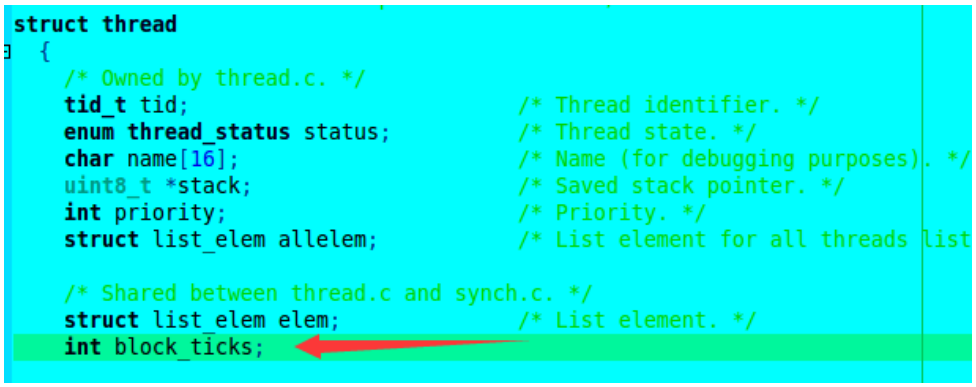
3.2 正式实验

根据实验目的，基于对实验内容的充分理解的基础上，我们需要修改一下三个文件中的代码。

文件名	路径
thread.h	*/pintos/src/threads*
timer.c	*/pintos/src/devices*
thread.c	*/pintos/src/threads*
...	...

1. 在 *thread.h* 中修改结构体在 *thread.h* 中的 thread 结构体最后加入

   - `int block_ticks;`



```
struct thread
  {
    /* Owned by thread.c. */
    tid_t tid;                          /* Thread identifier. */
    enum thread_status status;          /* Thread state. */
    char name[16];                      /* Name (for debugging purposes). */
    uint8_t *stack;                     /* Saved stack pointer. */
    int priority;                       /* Priority. */
    struct list_elem allelem;           /* List element for all threads list

    /* Shared between thread.c and synch.c. */
    struct list_elem elem;              /* List element. */
    int block_ticks;
```

图 18

2. 在 *timer.c* 中修改 *timer_sleep()*

```
timer_sleep (int64_t ticks)
{
    if (ticks > 0)
    {
        enum intr_level old_level;
        struct thread *t;
        t=thread_current ();
        t->block_ticks = ticks;          //记录阻塞时间
        old_level = intr_disable();
        thread_block();                  //阻塞进程
        intr_set_level(old_level);
    }
}
```

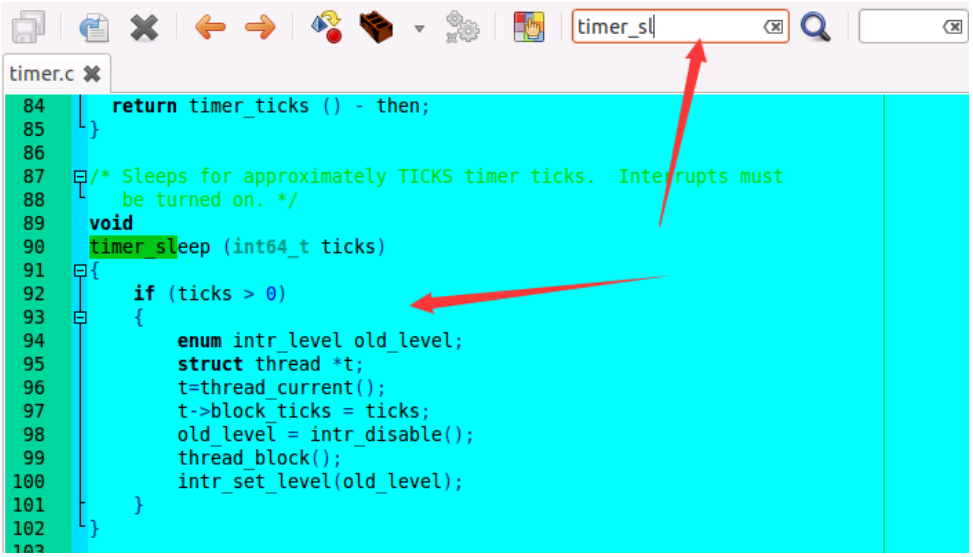


图 19

3. 定义 *block_check()* 函数

Step 1: 将 *block_check()* 加入到 *thread.h* 中

```
void block_check(struct thread *t, void *aux UNUSED);
```

Step 2: 在 *thread.c* 中对 *block_check()* 进行定义：

```
void block_check(struct thread *t, void *aux UNUSED)
{
    if (t->status == THREAD_BLOCKED && t->block_ticks>0)
    {
        t->block_ticks--;
        if (t->block_ticks == 0)
        {
            thread_unblock (t);
        }
    }
}
```

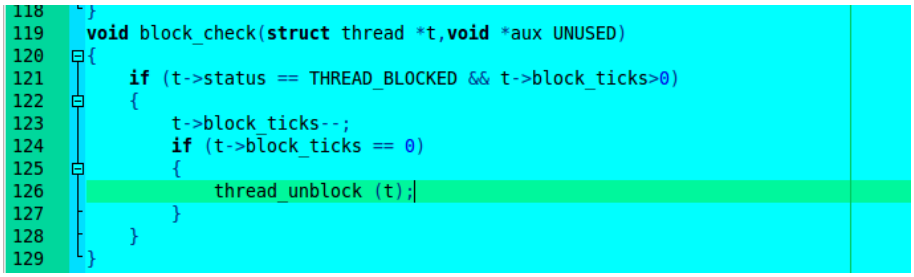


图 20

4. 在 *timer.c* 中修改调用 *block_check()*

```c
timer_interrupt (struct intr_frame *args UNUSED)
{
    ticks++;
    enum intr_level old_level;        //记录原来的中断状态
    old_level=intr_disable();         //thread_foreach函数要求关中断
    thread_foreach (block_check, NULL);
    intr_set_level (old_level);           //恢复中断
    thread_tick ();
}
```

3.3 实验结果

在 *teriminal* 中进入*../pintos/src/threads/*目录，输入：

```
make check
```

观察到只有以下 7 个检测均通过，其余均为不通过则为实验成功！

- *Alarm-single*
- *Alarm-multiple*
- *Alarm-simultaneous*
- *Alarm-zero*
- *Amarm-negative*
- *Mlfqs-fair-2*
- *Mlfqs-fair-20*

如下图所示

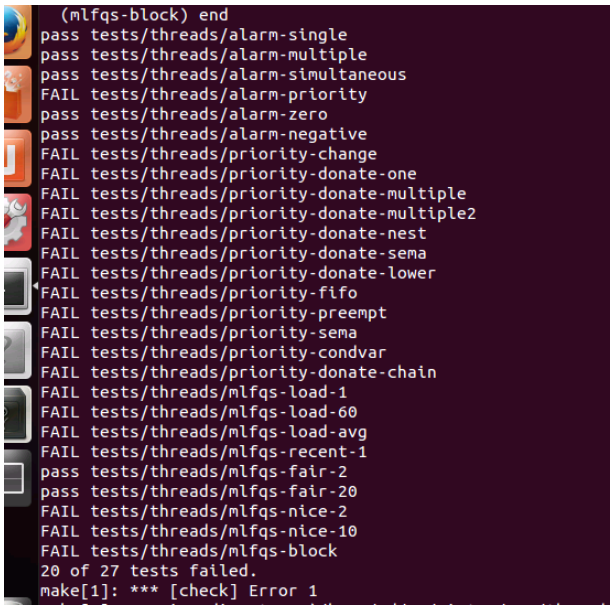


图 21: 结果截图

4 实验心得

通过本次实验，我粗略地了解了 *pintos* 的简单内核代码的含义，掌握了定时器中断的发生机制，本次实验是通过改变 *timer_sleep()* 函数，当执行该函数时，则将该进程阻塞，然后记录阻塞时间，在每一次发生定时器中断时才去检查 *wait_list,* 如果满足 (大于等于) 原定阻碍时间，则将其唤醒进入 *ready_list,* 显然这样做会使某些进程阻塞时间超过原定阻塞时间，所以存在进程无法 *pass*。